

DeferKV: Rethinking Eviction Timing for One-Shot KV Cache Compression

Zhe Wang^{1,2} Jiakai Li^{1,2} Yujia Sun^{1,2} Rongzheng Wang^{1,2} Shuang Liang^{1,2*}

¹University of Electronic Science and Technology of China, Chengdu, China

²Ubiquitous Intelligence and Trusted Services Key Laboratory of Sichuan Province, Chengdu, China

Abstract

Long-context large language models (LLMs) have demonstrated strong capabilities across a wide range of tasks, but the growing KV cache introduces substantial memory and inference overhead. Existing one-shot KV cache compression methods typically commit to irreversible eviction immediately after prefill, before any signal from actual generation becomes available. Our quantitative analysis shows that early queries from the actual generation stage provide attention signals that are more consistent with subsequent decode attention, with the largest single-step gain occurring at the prefill–decode boundary. Based on this observation, we propose DeferKV, which moves the eviction decision from the end of prefill to the first real decoding step and temporally combines prompt-side and decode-side observations, thereby better aligning KV importance estimation with subsequent generation requirements. DeferKV requires no additional training, draft model, or future-query prediction module, making it simple and easy to deploy. Experiments on LongBench, RULER, and Needle-in-a-Haystack demonstrate that DeferKV consistently improves model performance under KV cache compression while maintaining low inference latency.

become a major bottleneck affecting memory consumption and inference throughput (Li et al., 2025; Shi et al., 2024; Yuan et al., 2024). For autoregressive Transformer decoders (Vaswani et al., 2017), the key and value representations of historical tokens must be cached at every layer to avoid redundant computation during generation. As the context length increases, the memory consumption of the KV cache grows linearly with the sequence length, making long-context inference more challenging on memory-constrained devices and in high-concurrency serving scenarios. Therefore, compressing the KV cache while preserving generation quality has become an important research problem in efficient LLM inference.

Existing KV cache compression methods often perform one-time KV eviction immediately after the prefill stage. Representative methods such as SnapKV (Li et al., 2024) use a small set of queries near the end of the prompt as an observation window and estimate the importance of historical tokens based on the attention scores between these queries and historical keys, thereby selecting the KV tokens to retain. These methods are simple to implement, require no additional training, and offer favorable efficiency in practice.

However, these methods (Li et al., 2024; Liu et al., 2025) typically rely on a key assumption: the queries within the observation window at the end of the prompt are sufficient to reflect the actual attention requirements of subsequent decoding. This assumption does not always hold in long-text generation tasks. As shown in Figure 1, more historical KV tokens lie closer to the diagonal when using the first real decode query. Its attention scores are therefore more consistent with subsequent decode attention than those aggregated from the final 32 prefill queries. This suggests that prompt-end attention alone may not fully capture the access patterns observed during subsequent generation. Consequently, estimating KV importance solely

1 Introduction

Large language models (LLMs) with long-context capabilities (OpenAI, 2023; Team, 2025) have demonstrated strong performance in document question answering (Dasigi et al., 2021), multi-document summarization (Fabbri et al., 2019), code understanding (Bai et al., 2024), retrieval-augmented generation (Lewis et al., 2020), and complex reasoning (Yao et al., 2023). However, long-context inference also introduces substantial system overhead. In particular, the KV cache has

*Corresponding author: shuangliang@uestc.edu.cn

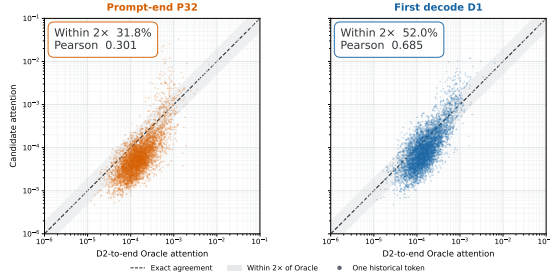


Figure 1: Consistency of prompt-end and first-decode attention with subsequent decode attention on GovReport using Qwen3-8B. Each point denotes a historical KV token. The horizontal axis shows its average attention from D_2 to the end of generation, while the vertical axis shows the average over the last 32 prefill queries and the attention from the first decode query. The dashed diagonal marks exact agreement, and the shaded region indicates values within a factor of two.

from prompt-side queries may lead to suboptimal token selection for later decoding.

Recent methods have begun to address this issue by incorporating information related to future generation into KV token importance estimation. For example, SpecKV (Galim et al., 2026), LAQ (Wang et al., 2025), and LookaheadKV (Ahn et al., 2026) construct or predict future queries or attention patterns to make KV compression more closely aligned with the requirements of the actual generation stage. Although these methods alleviate the limitations of prompt-only observation from different perspectives, they usually require additional models, auxiliary modules, or extra training procedures and inevitably face uncertainty introduced by prediction errors.

Based on these observations, we propose a more direct and lightweight approach. Unlike existing methods that rely solely on prompt-side information, our strategy postpones irreversible KV eviction until the first real decode-side signal becomes available and incorporates a small amount of attention feedback from actual generation. This explicitly aligns the importance estimation of historical tokens with the requirements of subsequent decoding. We further consider how to effectively integrate observations from the input and generation stages. To this end, we introduce a simple yet effective temporal weighting strategy to improve the reliability and accuracy of importance estimation.

Our main contributions are summarized as follows:

- We revisit the timing assumption of exist-

ing KV cache compression methods that perform eviction immediately after prefill. We propose a compression approach that postpones irreversible KV eviction until the earliest real decode-side feedback becomes available, thereby better aligning KV selection with subsequent decoding.

- Our method is simple, direct, and lightweight. It does not require additional training, a draft model, an auxiliary network, or a future-query prediction module. It only requires minor modifications to the existing inference process, enabling straightforward implementation and deployment.
- Extensive experiments show that our method consistently outperforms state-of-the-art baselines under constrained KV cache budgets, achieving relative performance gains of up to 3.58% on LongBench and 8.03% on RULER.

2 Related Work

2.1 KV Cache Compression and Token Eviction

To reduce the memory consumption of the KV cache, existing studies (Zhang et al., 2023; Liu et al., 2023) have explored compression by reducing the number of cached tokens. These methods typically retain important Key-Value pairs according to the potential contribution of historical tokens to subsequent generation, while evicting KV entries with lower importance, thereby controlling the cache size during decoding. Prompt-only compression methods usually perform a one-time selection of prompt KV tokens after the prefill stage and before decoding begins (Li et al., 2025). They are generally simple to implement and introduce limited scheduling overhead.

SnapKV (Li et al., 2024) uses an observation window near the end of the prompt to estimate the importance of historical KV tokens and selects clustered important KV positions within each attention head. PyramidKV (Cai et al., 2024) further considers layer-wise budget allocation and assigns larger cache budgets to lower layers and smaller budgets to higher layers according to their different information aggregation characteristics. Ada-KV (Feng et al., 2025) focuses on the importance differences across attention heads and adaptively allocates more cache capacity to heads that have a greater influence on the model output.

ChunkKV (Liu et al., 2025) treats semantic chunks rather than individual tokens as the basic compression units and introduces layer-wise index reuse to preserve contextual integrity while reducing selection overhead.

These methods rely primarily on prompt-side observations and may not fully reflect the actual attention requirements of subsequent decoding.

2.2 Future-Aware KV Cache Compression

To alleviate the limitations of prompt-only observation, recent studies have begun to introduce future-aware signals. These methods improve KV selection by predicting or approximating future queries, making the compression strategy more closely aligned with the requirements of the actual generation stage.

SpecKV (Galim et al., 2026) uses a lightweight draft model to generate approximate future tokens and employs these lookahead tokens to help the target model estimate the importance of prompt KV pairs for subsequent generation.

LAQ (Wang et al., 2025) performs low-budget lookahead decoding after prefill to generate a small number of pseudo-response tokens. It then caches their corresponding query states as a Q-Cache and uses them to re-evict the KV cache.

LookaheadKV (Ahn et al., 2026) further introduces learnable lookahead tokens and combines them with LoRA (Hu et al., 2022) modules to predict future attention patterns, thereby obtaining future-aware signals for KV selection.

Although these methods better align KV compression with generation requirements, they generally introduce additional model components, training procedures, or inference overhead, increasing system complexity and deployment cost.

3 Observations

To characterize the transition from prompt-side observations to actual decode-stage access patterns, we compare the attention distributions produced by queries at different positions with a reference attention distribution constructed from later decode queries. We consider both distributional similarity and the overlap between their induced KV-token selections.

Specifically, we evaluate the final 32 prefill queries and the first 8 real decode queries, with relative positions

$$r \in \{-32, -31, \dots, -1, 1, 2, \dots, 8\}. \quad (1)$$

Experiments are conducted using Qwen3-8B (Team, 2025), Llama-3.1-8B-Instruct (Team, 2024), and Mistral-7B-Instruct-v0.3 (Jiang et al., 2023) on GovReport (Huang et al., 2021), MultiNews (Fabbri et al., 2019), QMSum (Zhong et al., 2021), and RepoBench-P (Liu et al., 2024b). For each model–task combination, we randomly sample 30 examples, resulting in 360 evaluation instances in total. The KV budget is set to $B = 512$.

For each sample, we perform the complete generation process under the FullKV setting. Let N_n and T_n denote the prompt length and the number of generated tokens for sample n , respectively. To ensure a fair comparison, all queries are evaluated over the same compressible prompt prefix, which consists of the prompt positions preceding the final 32 tokens. For each query at relative position r , its attention weights over this prefix are renormalized to form the distribution $\mathbf{p}_{n,\ell,h}^{(r)}$ at layer ℓ and attention head h .

Let $\mathbf{a}_{n,\ell,h}^{(t)}$ denote the normalized attention distribution of the t -th decode query over the same prompt prefix. We construct the reference attention distribution by averaging the normalized attention distributions of the decode queries from step 9 to the final generation step:

$$\mathbf{o}_{n,\ell,h} = \frac{1}{T_n - 8} \sum_{t=9}^{T_n} \mathbf{a}_{n,\ell,h}^{(t)}. \quad (2)$$

3.1 Metrics

We evaluate the consistency between each query position and the reference attention distribution from two perspectives.

JSD-Based Similarity. The Jensen–Shannon divergence (JSD) (Lin, 1991) is a symmetric measure of the difference between two probability distributions. For the query attention distribution $\mathbf{p}$ and the reference distribution $\mathbf{o}$, let $\mathbf{m} = (\mathbf{p} + \mathbf{o})/2$. Their JSD is defined as

$$\text{JSD}(\mathbf{p}, \mathbf{o}) = \frac{1}{2} D_{\text{KL}}(\mathbf{p} \parallel \mathbf{m}) + \frac{1}{2} D_{\text{KL}}(\mathbf{o} \parallel \mathbf{m}). \quad (3)$$

With base-2 logarithms, JSD lies in $[0, 1]$, where a smaller value indicates greater similarity between the two attention distributions. To facilitate trend comparison, we further define a JSD-based similarity score as

$$\text{JSDSim}_{n,\ell,h}^{(r)} = 1 - \text{JSD}(\mathbf{p}_{n,\ell,h}^{(r)}, \mathbf{o}_{n,\ell,h}). \quad (4)$$

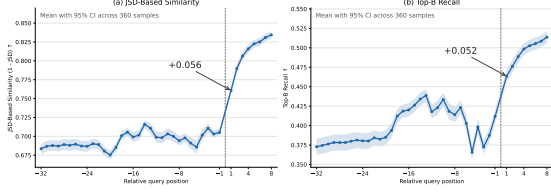


Figure 2: JSD-Based Similarity and Top- B KV Selection Recall at different query positions.

A higher value indicates greater consistency between the query attention distribution and the reference attention distribution.

KV Selection Recall. For sample n , layer ℓ , and attention head h , let $\mathcal{S}_{n,\ell,h}^*$ denote the Top- B prompt KV positions selected according to the reference attention distribution, and let $\mathcal{S}_{n,\ell,h}^{(r)}$ denote those selected according to the attention scores of the query at relative position r . The corresponding KV Selection Recall is defined as

$$\text{Recall}_{n,\ell,h}^{(r)} = \frac{|\mathcal{S}_{n,\ell,h}^{(r)} \cap \mathcal{S}_{n,\ell,h}^*|}{B}. \quad (5)$$

For each query position, we first average the metrics over all layers and attention heads within each sample and then average them over all 360 evaluation instances. The shaded regions in Figure 2 indicate 95% confidence intervals.

3.2 Obs. 1: Prefill-to-Decode Attention Shift

As shown in Figure 2, prefill queries already show moderate consistency with the reference attention distribution, indicating that prompt-side attention contains useful signals for KV importance estimation (Li et al., 2024). However, moving from the final prefill query at $r = -1$ to the first real decode query at $r = 1$ increases JSD-Based Similarity and KV Selection Recall by 0.056 and 0.052, respectively.

This sharp improvement indicates that the first real decode query reveals generation-specific access requirements that are not fully captured during prefill, making it a more reliable signal for KV importance estimation.

3.3 Obs. 2: Largest Gain at First Decode

Among all adjacent query positions, the largest single-step gain occurs at the prefill–decode boundary. In comparison, the average gains between adjacent subsequent decode positions are only 0.009

for JSD-Based Similarity and 0.007 for KV Selection Recall, both substantially lower than those at the boundary.

Later decode queries can further improve consistency with the reference attention distribution. However, using these later queries requires retaining the full prompt KV cache for additional decode steps, thereby increasing memory usage and end-to-end inference latency. Therefore, the first real decode query provides the earliest decode-side observation with the largest gain, offering a favorable trade-off between KV selection quality and system overhead.

3.4 Obs. 3: Temporal Relevance Decay

Beyond the improvement at the prefill–decode boundary, both metrics generally increase as query positions approach the decode stage. Although the trend is not strictly monotonic, earlier prefill queries are typically less consistent with the reference attention distribution.

This suggests that queries contribute differently to KV importance estimation. We therefore assign larger weights to queries closer to the decode stage and gradually discount earlier queries, emphasizing generation-related signals while preserving prompt-side context.

4 Method

4.1 Overall Workflow

Prompt-side attention does not fully reflect the KV access requirements of subsequent decoding, while the first real decode query provides a more reliable signal and queries closer to decoding are generally more informative. Based on these observations, DeferKV postpones KV cache compression until after the first standard decoding step.

As illustrated in Figure 3, DeferKV obtains the first real decode query from the target model and combines it with recent prompt queries to form a decode-aware observation window. Their attention scores are aggregated using temporal weighting, assigning greater importance to queries closer to the decode stage. The resulting scores are used to estimate prompt-token importance and construct the compressed KV cache for subsequent generation. In this way, DeferKV better aligns KV selection with actual decoding requirements without predicting future queries, introducing auxiliary models, or altering the standard generation trajectory.

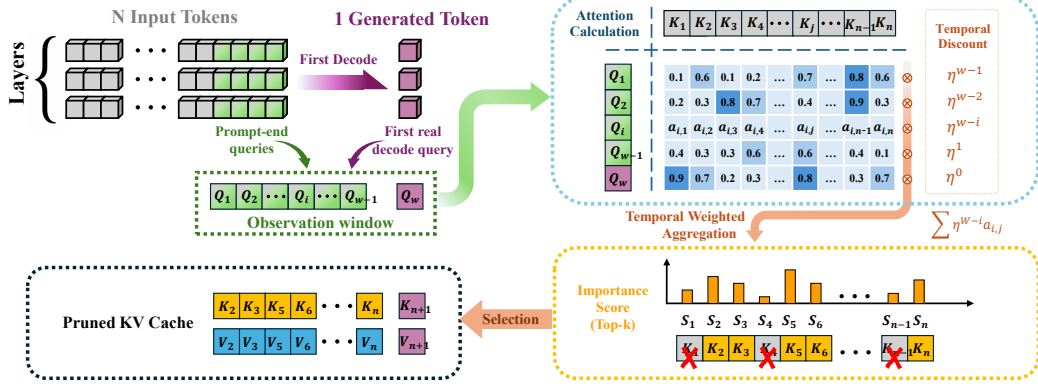


Figure 3: Overview of DeferKV. The first real decode query is combined with recent prompt queries and temporally weighted to estimate KV importance for subsequent cache compression.

4.2 First Real Decode Query Integration

Let $\mathbf{X} = \{x_1, x_2, \dots, x_N\}$ denote a tokenized input prompt, where x_i denotes the token at position i and N denotes the prompt length. The output logits at the final prompt position define the conditional distribution of the first generated token. Accordingly, the first token is obtained as

$$y_1 = \text{Decode}(p_\theta(\cdot | \mathbf{X})), \quad (6)$$

where p_θ denotes the target model and $\text{Decode}(\cdot)$ denotes the adopted decoding strategy, such as greedy decoding or sampling.

The generated token y_1 is then fed into the model for the first standard decoding step. At layer ℓ , the corresponding Query, Key, and Value representations are denoted by $\mathbf{q}_{\text{dec},1}^\ell$, $\mathbf{k}_{\text{dec},1}^\ell$, and $\mathbf{v}_{\text{dec},1}^\ell$, respectively.

Here, $\mathbf{q}_{\text{dec},1}^\ell$ is the real decode query produced by the target model when processing the first generated token. Given an observation window of size W , we combine this query with the last $W - 1$ prompt queries to construct a new observation window:

$$\mathbf{Q}_{\text{obs}}^\ell = [\mathbf{q}_{N-W+2}^\ell, \dots, \mathbf{q}_N^\ell, \mathbf{q}_{\text{dec},1}^\ell]. \quad (7)$$

This design preserves the original observation-window size while incorporating information from the actual generation stage into KV-token importance estimation.

4.3 Temporally Weighted Decode-Aware Eviction

Given the observation window defined above, we compute its attention weights over the prompt Keys as

$$\mathbf{A}^\ell = \text{Attention}(\mathbf{Q}_{\text{obs}}^\ell, \mathbf{K}^\ell). \quad (8)$$

where $\mathbf{K}^\ell$ denotes the prompt Keys at the ℓ -th layer. The attention computation preserves the original visibility constraints of the model. We denote the (i, j) -th element of $\mathbf{A}^\ell$ by $a_{i,j}^\ell$, representing the attention weight assigned by the i -th observation query to the j -th prompt Key.

The queries in the observation window are ordered chronologically, from the earlier prompt queries to the first real decode query. Motivated by the temporal relevance decay in Observation 3, we assign larger weights to queries closer to the decode stage. The importance score of the j -th prompt token is computed as

$$s_j^\ell = \sum_{i=1}^W \eta^{W-i} a_{i,j}^\ell, \quad 0 < \eta \leq 1, \quad (9)$$

where η is the temporal discount factor. Therefore, the first real decode query receives a weight of 1, while earlier prompt queries are exponentially discounted according to their temporal distance from the decode stage. When $\eta = 1$, this aggregation is equivalent to mean aggregation in terms of token ranking.

After obtaining the importance scores, we retain the local smoothing, budget allocation, and cache selection strategies of the underlying KV cache eviction method (Li et al., 2024), while replacing its original prompt-only importance scores with the proposed decode-aware scores. The Key and Value corresponding to the first generated token are then combined with the compressed prompt KV cache to construct the KV cache used for subsequent decoding.

The model thereafter continues standard autoregressive generation. Our method only changes the timing of the initial KV cache compression and

reuses the query naturally produced during the target model’s decoding process. Therefore, it requires no modification to the model architecture or additional training procedure.

5 Experiments

5.1 Experimental Setup

Benchmarks and Models. We conduct experiments on LongBench (Bai et al., 2024), RULER (Hsieh et al., 2024), and Needle-in-a-Haystack (Liu et al., 2024a). LongBench contains 16 long-context datasets across six task categories, covering question answering, summarization, few-shot learning, synthetic tasks, and code understanding. RULER evaluates model capabilities on challenging long-context tasks across varying context lengths. Needle-in-a-Haystack assesses the ability to retrieve critical information from long sequences, with detailed results reported in Appendix F.

We evaluate Qwen3-8B (Team, 2025), Llama-3.1-8B-Instruct (Team, 2024), and Mistral-7B-Instruct-v0.3 (Jiang et al., 2023). For each model, we use its official tokenizer and prompt template and adopt greedy decoding.

Selected Baselines. We compare against four representative KV cache compression methods: (1) **SnapKV** (Li et al., 2024), which estimates historical KV importance using a local observation window near the end of the prompt and performs one-time compression before decoding; (2) **LAQ** (Wang et al., 2025), which performs low-budget lookahead decoding with an initially compressed cache and uses the resulting query states to guide a second KV eviction; (3) **SpecKV** (Galim et al., 2026), which uses a small draft model to generate a proxy response and processes it with the target model to estimate prompt-KV importance; (4) **LookaheadKV** (Ahn et al., 2026), which uses learnable lookahead tokens and lightweight LoRA modules to estimate prompt-KV importance.

Implementation Details. We evaluate DeferKV and all baselines under varying cache budgets ($B_{\text{total}} = nL$, with $n \in [64, 1024]$), where n is the prompt KV budget for each layer and L is the number of layers. We use greedy decoding without sampling for all experiments to ensure deterministic and reproducible evaluation. We use the official baseline implementations with standard eviction settings, with further details provided in

Appendix B. All experiments are conducted on NVIDIA A100 GPUs with 80 GB of memory.

5.2 Results on LongBench

We evaluate DeferKV on 16 LongBench datasets using three language models. Table 1 reports detailed results under a tight cache budget of $B_{\text{total}} = 64L$ and a larger cache budget of $B_{\text{total}} = 512L$. DeferKV achieves the highest average score across all six model–budget settings.

Under the tight $64L$ budget, DeferKV consistently ranks as the top-performing method across models, achieving up to a 3.58% relative improvement over the strongest baseline with Qwen3-8B. This advantage is particularly pronounced under constrained cache budgets, where real decode feedback improves KV importance estimation and helps preserve information essential for subsequent generation.

As the budget increases to $B_{\text{total}} = 512L$, the performance gaps among methods narrow because more prompt KV tokens can be retained. Nevertheless, DeferKV continues to achieve the best average performance and approaches Full KV across all three models. In addition to its strong performance, DeferKV requires no additional training, draft model, or future-query prediction module, making it simple to implement and deploy. We further integrate DeferKV with PyramidKV, AdaKV, and ChunkKV to demonstrate its compatibility with existing KV cache compression methods. The corresponding results are provided in Appendix D.

5.3 Results on RULER

We evaluate DeferKV on 11 tasks from the RULER benchmark using Llama-3.1-8B-Instruct, with a fixed total cache budget of $B_{\text{total}} = 1024L$ applied across all methods. Table 2 summarizes the average accuracy across context lengths ranging from 4K to 128K. Existing KV cache eviction methods suffer from substantial performance degradation as the context length increases, highlighting their limited robustness in ultra-long-context and complex retrieval scenarios.

In contrast, DeferKV consistently achieves strong performance across all context lengths, improving the average accuracy by 8.03% over the strongest baseline. Notably, even at a context length of 128K, where less than 1% of the original KV cache is retained, DeferKV maintains effective retrieval capability. These results demonstrate that incorporating a real decode query provides a more

Method	Single-Doc QA			Multi-Doc QA			Summ.			Few-Shot			Synth.		Code		Avg.
	NrrvQA	Qasper	MF-en	HotpotQA	2WikiMQA	MuSiQue	GovReport	QMSum	MultiNews	TREC	TriviaQA	SAMSum	PCount	PR-en	LCC	RB-P	
Qwen3-8B, $B_{\text{total}} = 64L$																	
SnapKV	17.41	30.06	38.05	47.77	37.71	20.65	14.00	19.37	12.93	43.50	81.92	36.53	5.00	87.50	59.30	53.58	37.83
LAQ	18.99	37.88	43.31	<u>53.88</u>	<u>41.35</u>	23.24	16.18	19.40	<u>14.78</u>	<u>66.00</u>	<u>88.40</u>	37.64	5.50	94.50	<u>62.31</u>	<u>55.91</u>	<u>42.45</u>
SpecKV	<u>19.61</u>	29.97	49.79	51.03	36.97	22.14	13.27	19.72	14.82	37.50	73.63	36.71	6.00	<u>95.00</u>	45.15	46.62	37.37
LookaheadKV	18.22	32.27	40.12	47.66	38.01	<u>23.58</u>	14.61	18.36	13.18	46.95	83.29	36.69	5.00	<u>90.40</u>	59.06	54.28	38.86
DeferKV	21.87	<u>35.37</u>	<u>45.97</u>	56.99	41.88	31.20	<u>14.81</u>	<u>19.52</u>	13.52	70.50	89.98	<u>37.44</u>	8.00	97.50	62.52	56.47	43.97
Qwen3-8B, $B_{\text{total}} = 512L$																	
SnapKV	22.70	42.96	51.19	57.09	42.51	34.94	25.14	23.44	21.87	68.50	89.46	43.34	9.50	100.00	68.74	64.22	47.85
LAQ	23.16	43.58	<u>52.63</u>	56.16	43.02	34.36	25.46	22.02	21.94	<u>71.00</u>	90.05	43.35	9.50	100.00	67.92	<u>64.72</u>	48.05
SpecKV	<u>24.38</u>	43.27	53.93	<u>58.07</u>	42.90	31.48	29.51	<u>23.24</u>	24.05	<u>71.00</u>	89.96	29.60	4.00	<u>99.00</u>	56.84	56.85	46.13
LookaheadKV	23.10	<u>43.67</u>	52.21	56.86	<u>43.12</u>	<u>34.91</u>	25.46	22.71	22.08	<u>71.00</u>	<u>89.97</u>	43.85	9.50	<u>99.00</u>	<u>68.01</u>	64.98	<u>48.15</u>
DeferKV	24.82	44.78	51.96	58.09	43.18	33.83	<u>26.04</u>	22.74	<u>22.17</u>	71.50	90.05	<u>43.70</u>	9.50	100.00	<u>67.55</u>	<u>64.02</u>	48.37
Full KV	24.73	48.05	54.39	58.16	43.79	34.27	33.45	24.12	24.84	71.50	90.21	44.56	9.50	100.00	69.34	65.48	49.77
Llama-3.1-8B-Instruct, $B_{\text{total}} = 64L$																	
SnapKV	27.32	25.25	44.30	53.98	42.48	27.10	17.88	22.20	17.45	41.00	85.37	37.72	<u>8.50</u>	100.00	55.35	48.69	40.91
LAQ	27.66	<u>28.03</u>	47.05	58.12	<u>45.06</u>	<u>30.60</u>	18.85	22.51	<u>19.01</u>	71.50	90.02	40.16	<u>8.50</u>	<u>99.50</u>	56.39	<u>51.29</u>	<u>44.64</u>
SpecKV	<u>29.31</u>	23.34	56.27	56.81	41.91	28.70	21.22	24.43	18.66	46.98	76.62	39.69	9.71	100.00	41.67	42.44	41.11
LookaheadKV	27.95	26.61	47.23	54.93	44.90	30.10	19.24	22.01	18.34	<u>70.00</u>	88.65	<u>40.21</u>	<u>8.50</u>	<u>99.00</u>	<u>56.54</u>	49.03	43.95
DeferKV	29.83	31.02	<u>49.95</u>	<u>57.55</u>	47.65	30.77	<u>20.68</u>	<u>22.77</u>	19.29	71.50	<u>88.95</u>	40.41	7.88	100.00	57.98	51.56	45.49
Llama-3.1-8B-Instruct, $B_{\text{total}} = 512L$																	
SnapKV	29.92	40.20	55.21	57.85	48.02	30.03	25.79	<u>24.40</u>	24.33	65.00	91.95	41.80	8.08	<u>99.50</u>	64.39	<u>57.27</u>	47.73
LAQ	<u>30.58</u>	<u>42.99</u>	54.46	58.13	48.40	30.79	26.45	24.15	24.51	<u>70.96</u>	91.51	<u>42.74</u>	7.59	<u>99.50</u>	<u>64.08</u>	56.78	<u>48.35</u>
SpecKV	28.23	41.70	56.69	59.29	<u>48.62</u>	28.24	30.41	26.44	27.08	72.00	91.63	28.51	3.23	<u>99.00</u>	53.07	49.99	46.51
LookaheadKV	30.11	41.12	<u>55.34</u>	57.98	49.23	31.34	26.02	23.85	24.06	72.00	<u>91.85</u>	41.64	<u>7.75</u>	<u>99.50</u>	63.21	57.33	48.27
DeferKV	30.70	43.91	54.84	<u>58.54</u>	48.51	<u>31.02</u>	<u>26.60</u>	24.22	<u>24.53</u>	72.00	91.48	42.98	7.47	100.00	63.90	56.70	48.59
Full KV	31.20	46.66	56.64	58.10	48.51	31.57	34.37	25.18	27.14	73.00	91.65	43.80	7.64	99.50	65.18	58.90	49.94
Mistral-7B-Instruct-v0.3, $B_{\text{total}} = 64L$																	
SnapKV	17.65	22.08	36.24	41.95	33.98	20.89	18.83	21.14	16.34	38.50	86.96	39.81	4.50	77.00	51.57	49.59	36.06
LAQ	19.72	<u>28.68</u>	40.14	<u>48.46</u>	<u>35.96</u>	<u>23.44</u>	<u>20.82</u>	21.47	18.93	<u>70.58</u>	89.15	42.32	4.75	79.80	<u>53.08</u>	<u>51.19</u>	<u>40.53</u>
SpecKV	<u>19.74</u>	21.08	48.10	45.00	33.32	22.44	22.64	22.93	17.89	38.49	78.44	40.89	<u>5.60</u>	<u>80.75</u>	37.66	42.98	36.12
LookaheadKV	19.48	24.28	38.88	42.86	34.76	23.22	<u>20.82</u>	21.02	18.93	57.38	88.09	41.11	4.50	78.16	51.92	49.93	38.46
DeferKV	26.11	28.83	<u>43.02</u>	49.10	36.87	25.15	20.49	<u>21.75</u>	<u>18.50</u>	73.50	<u>89.04</u>	<u>42.17</u>	7.50	81.00	53.41	51.46	41.74
Mistral-7B-Instruct-v0.3, $B_{\text{total}} = 512L$																	
SnapKV	25.48	33.15	<u>49.37</u>	48.98	36.54	24.68	25.61	<u>24.27</u>	24.11	67.50	<u>89.36</u>	44.82	6.00	97.50	60.36	<u>61.28</u>	44.94
LAQ	<u>26.23</u>	<u>34.50</u>	48.82	49.95	38.22	26.58	26.10	24.10	24.20	74.66	89.08	<u>47.10</u>	6.00	97.50	<u>60.02</u>	61.44	<u>45.91</u>
SpecKV	25.48	34.10	51.48	<u>50.19</u>	37.03	22.06	30.10	25.69	25.58	72.25	89.45	31.30	<u>3.83</u>	<u>96.75</u>	48.75	53.96	43.63
LookaheadKV	25.79	33.86	49.15	48.94	39.16	<u>26.67</u>	25.88	24.12	23.93	<u>75.29</u>	89.20	46.33	6.00	97.50	59.54	61.44	45.80
DeferKV	26.90	35.37	48.91	51.10	<u>38.72</u>	27.63	<u>26.44</u>	24.13	<u>24.27</u>	76.00	89.07	47.18	6.00	97.50	59.68	60.49	46.21
Full KV	27.01	37.97	50.14	51.69	39.17	28.02	34.27	25.66	26.62	76.00	88.59	47.50	7.00	98.00	61.53	62.59	47.61

Table 1: Performance comparison on 16 LongBench datasets. The best and second-best results among the five cache-compression methods are highlighted in bold and underlined, respectively. Full KV is included only as a reference.

Method	4K	8K	16K	32K	64K	128K	Avg.
Full KV	99.34	98.83	98.55	94.89	89.85	79.32	93.46
SnapKV	83.63	75.51	71.09	66.12	56.26	45.67	66.38
LAQ	87.42	81.78	78.27	74.09	68.23	62.06	75.31
SpecKV	86.14	79.43	75.60	70.53	62.83	54.17	71.45
LookaheadKV	86.66	80.69	76.98	72.43	65.73	59.26	73.63
DeferKV	92.72	86.29	83.89	80.31	75.89	69.04	81.36

Table 2: Performance comparison on the RULER benchmark across different context lengths.

effective generation-aware signal for selecting critical KV tokens in ultra-long-context scenarios.

5.4 Efficiency Analysis

Peak Memory and Inference Latency. We evaluate inference efficiency using peak GPU mem-

ory and total end-to-end latency. All methods use Qwen3-8B with a 32K-token input, 128 generated tokens, and a cache budget of $B_{\text{total}} = 1024L$. We report the average results over five runs after warm-up.

Figure 4 compares the peak GPU memory and end-to-end inference latency of different methods. DeferKV achieves a latency of 11.56s, comparable to SnapKV and lower than LAQ, SpecKV, and LookaheadKV. Its peak memory usage is 22.84 GiB, lower than LAQ and SpecKV, although higher than SnapKV and LookaheadKV. Overall, DeferKV provides a favorable balance between memory usage and inference latency.

Effect of Compression Delay Steps. We vary the number of real decode queries observed before

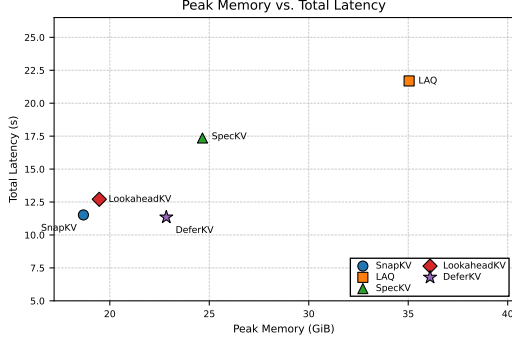


Figure 4: Peak GPU memory and end-to-end inference latency of different methods on Qwen3-8B.

Delay d	Score $\uparrow$	Peak Mem. $\downarrow$	Latency $\downarrow$
1	27.06	22.84	11.562
2	27.11	22.85	11.618
3	27.15	22.85	11.671
4	27.21	22.86	11.751

Table 3: Effect of compression delay steps on 32K-token GovReport samples using Qwen3-8B. Peak memory is reported in GiB, and latency is reported in seconds.

compression as $d \in \{1, 2, 3, 4\}$, where $d = 1$ is the default DeferKV setting. Larger values retain the full prompt KV cache for additional decoding steps before compression. All settings are evaluated on 100 randomly selected 32K-token samples from GovReport using Qwen3-8B, with 128 generated tokens and a cache budget of $B_{\text{total}} = 1024L$. Task scores are averaged over the 100 samples, while memory and latency are measured on a single sample and averaged over five runs after warm-up.

As shown in Table 3, delaying compression for additional decode steps yields only marginal performance gains. Larger d values keep the full prompt KV cache resident for longer and require more decoding steps before eviction, leading to increased end-to-end latency. We therefore adopt $d = 1$ as the default setting.

5.5 Ablation Study

We conduct ablation studies on LongBench to evaluate the contributions of real decode feedback and temporally weighted aggregation. We use Qwen3-8B with a cache budget of $B_{\text{total}} = 128L$.

As shown in Table 4, removing real decode feedback decreases the average score by 1.74, showing that the first real decode query provides useful information for KV importance estimation. Replacing temporal weighting with uniform aggregation reduces the score by 0.67, confirming the benefit

Method	Avg. Score $\uparrow$
DeferKV	46.12
w/o Decode Feedback	44.38 (−1.74)
w/o Temporal Weighting	45.45 (−0.67)

Table 4: Ablation results of DeferKV on LongBench using Qwen3-8B.

of emphasizing queries closer to the decode stage. Both components contribute to the effectiveness of DeferKV.

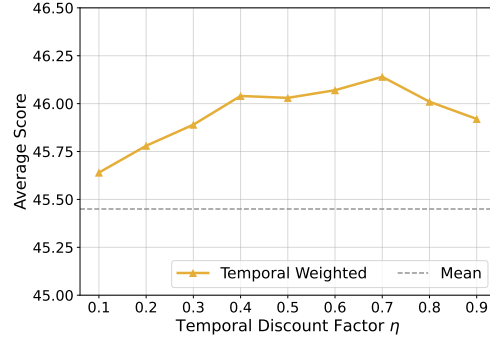


Figure 5: Effect of the temporal discount factor η on model performance.

5.6 Hyperparameter Sensitivity

We evaluate the effect of the temporal discount factor η on LongBench using Qwen3-8B with a cache budget of $B_{\text{total}} = 128L$. As shown in Figure 5, DeferKV maintains stable performance over a wide range of values. Based on these results, we uniformly set $\eta = 0.7$ for all models and tasks without additional tuning for specific scenarios.

6 Conclusion

In this paper, we revisit a basic design choice in one-shot KV cache compression: when to commit to irreversible eviction. DeferKV moves this decision to the first decoding step, where real generation-side feedback becomes available, and uses temporal weighting to integrate the first real decode query with recent prompt queries. Extensive evaluations on LongBench, Needle-in-a-Haystack, and RULER show that DeferKV improves long-context performance with low inference latency. It requires no additional training, draft model, or auxiliary prediction module, providing a lightweight approach to efficient long-context inference.

Limitations

DeferKV postpones one-shot KV eviction until the earliest real decode query becomes available, which requires retaining the full prompt KV cache through the first decoding step. This design therefore shifts the memory reduction to subsequent decoding. Our experiments focus on models with standard token-wise KV caches under softmax attention. Extending DeferKV to architectures with different cache organizations, such as MLA or hybrid linear-attention models, may require architecture-specific adaptation.

References

- Jinwoo Ahn, Ingyu Seong, Akhil Kedia, Junhan Kim, Hyemi Jang, Kangwook Lee, and Yongkweon Jeon. 2026. Lookaheadkv: Fast and accurate KV cache eviction by glimpsing into the future without generation. *CoRR*, abs/2603.10899.
- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. Longbench: A bilingual, multi-task benchmark for long context understanding. In *ACL*, pages 3119–3137.
- Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, and Wen Xiao. 2024. Pyramidkv: Dynamic KV cache compression based on pyramidal information funneling. *CoRR*, abs/2406.02069.
- Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. 2021. A dataset of information-seeking questions and answers anchored in research papers. In *NAACL-HLT*, pages 4599–4610.
- Alexander R. Fabbri, Irene Li, Tianwei She, Suyi Li, and Dragomir R. Radev. 2019. Multi-news: A large-scale multi-document summarization dataset and abstractive hierarchical model. In *ACL*, pages 1074–1084.
- Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S. Kevin Zhou. 2025. Ada-kv: Optimizing KV cache eviction by adaptive budget allocation for efficient LLM inference. In *NeurIPS*.
- Kevin Galim, Ethan Ewer, Wonjun Kang, Minjae Lee, Hyung Il Koo, and Kangwook Lee. 2026. Draft-based approximate inference for LLMs. In *The Fourteenth International Conference on Learning Representations*.
- Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekish, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: what’s the real context size of your long-context language models? *CoRR*, abs/2404.06654.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. Lora: Low-rank adaptation of large language models. In *ICLR*.
- Luyang Huang, Shuyang Cao, Nikolaus Nova Parulian, Heng Ji, and Lu Wang. 2021. Efficient attentions for long document summarization. In *NAACL-HLT*, pages 1419–1436.
- Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. 2023. Mistral 7b. *CoRR*, abs/2310.06825.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich K  ttler, Mike Lewis, Wen-tau Yih, Tim Rock  tschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*.
- Haoyang Li, Yiming Li, Anxin Tian, Tianhao Tang, Zhanchao Xu, Xuejia Chen, Nicole Hu, Wei Dong, Qing Li, and Lei Chen. 2025. A survey on large language model acceleration based on KV cache management. *Trans. Mach. Learn. Res.*, 2025.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. Snapkv: LLM knows what you are looking for before generation. In *NeurIPS*.
- Jianhua Lin. 1991. Divergence measures based on the shannon entropy. *IEEE Trans. Inf. Theory*, 37(1):145–151.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024a. Lost in the middle: How language models use long contexts. *Trans. Assoc. Comput. Linguistics*, 12:157–173.
- Tianyang Liu, Canwen Xu, and Julian J. McAuley. 2024b. Repobench: Benchmarking repository-level code auto-completion systems. In *ICLR*.
- Xiang Liu, Zhenheng Tang, Peijie Dong, Zeyu Li, Bo Li, Xuming Hu, and Xiaowen Chu. 2025. Chunkkv: Semantic-preserving KV cache compression for efficient long-context LLM inference. *CoRR*, abs/2502.00299.
- Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhao Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. 2023. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *NeurIPS*.
- OpenAI. 2023. GPT-4 technical report. *CoRR*, abs/2303.08774.

Luohe Shi, Hongyi Zhang, Yao Yao, Zuchao Li, and Hai Zhao. 2024. Keep the cost down: A review on methods to optimize llm’s kv-cache consumption. *CoRR*, abs/2407.18003.

Llama Team. 2024. The llama 3 herd of models. *CoRR*, abs/2407.21783.

Qwen Team. 2025. Qwen3 technical report. *CoRR*, abs/2505.09388.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *NeurIPS*, pages 5998–6008.

Yixuan Wang, Shiyu Ji, Yijun Liu, Yuzhuang Xu, Yang Xu, Qingfu Zhu, and Wanxiang Che. 2025. Lookahead q-cache: Achieving more consistent KV cache eviction via pseudo query. In *EMNLP*, pages 34158–34174.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *ICLR*.

Jiayi Yuan, Hongyi Liu, Shaochen (Henry) Zhong, Yuning Chuang, Songchen Li, Guanchu Wang, Duy Le, Hongye Jin, Vipin Chaudhary, Zhaozhuo Xu, Zirui Liu, and Xia Ben Hu. 2024. KV cache compression, but what must we give in return? A comprehensive benchmark of long context capable approaches. In *EMNLP*, volume EMNLP 2024, pages 4623–4648.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark W. Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: heavy-hitter oracle for efficient generative inference of large language models. In *NeurIPS*.

Ming Zhong, Da Yin, Tao Yu, Ahmad Zaidi, Mutethia Mutuma, Rahul Jha, Ahmed Hassan Awadallah, Asli Celikyilmaz, Yang Liu, Xipeng Qiu, and Dragomir R. Radev. 2021. Qmsum: A new benchmark for query-based multi-domain meeting summarization. In *NAACL-HLT*, pages 5905–5921.

A Additional Details of Cache Selection and Construction

After obtaining the decode-aware importance scores s_j^ℓ , DeferKV retains the local smoothing, budget allocation, and cache selection strategies of the underlying KV cache eviction method, replacing only its original prompt-only importance scores.

We define the historical candidate set as $\mathcal{C} = \{1, \dots, N - W + 1\}$ and the protected recent set as $\mathcal{R} = \{N - W + 2, \dots, N\}$. The positions in $\mathcal{C}$ participate in importance-based selection, while

the last $W - 1$ prompt positions in $\mathcal{R}$ are retained directly.

Let s^ℓ denote the ordered vector of importance scores over $\mathcal{C}$. Following the underlying eviction method, average pooling with kernel size k is applied as

$$\bar{s}^\ell = \text{AvgPool}(s^\ell, k). \quad (10)$$

Let B denote the prompt-KV budget assigned to layer ℓ by the underlying budget-allocation strategy. Since $\mathcal{R}$ contains $W - 1$ protected positions, we select $B - W + 1$ positions from $\mathcal{C}$:

$$\mathcal{H}^\ell = \text{TopK}(\bar{s}^\ell, B - W + 1). \quad (11)$$

Here, $\mathcal{H}^\ell$ contains the corresponding prompt-position indices. The complete set of retained prompt positions is

$$\mathcal{I}^\ell = \text{Sort}(\mathcal{H}^\ell \cup \mathcal{R}), \quad (12)$$

which contains exactly B prompt positions.

Let $\mathbf{k}^{\ell,1}$ and $\mathbf{v}^{\ell,1}$ denote the Key and Value produced at the first decode step. The retained prompt KV pairs and the first decode KV pair are combined as

$$\mathbf{K}^\ell \leftarrow [\mathbf{K}^\ell[\mathcal{I}^\ell]; \mathbf{k}^{\ell,1}], \quad (13)$$

$$\mathbf{V}^\ell \leftarrow [\mathbf{V}^\ell[\mathcal{I}^\ell]; \mathbf{v}^{\ell,1}], \quad (14)$$

where the semicolon denotes concatenation along the sequence dimension. Standard autoregressive generation then continues using the updated KV cache.

B Implementation Details

DeferKV. We use an observation-window size of $W = 32$, a temporal discount factor of $\eta = 0.7$, and average pooling with kernel size 5. Compression is performed after the first standard decode step ($d = 1$). The aggregate prompt-KV budget is $B_{\text{total}} = nL$, where n is the retained prompt-KV budget per layer and L is the number of model layers.

SnapKV. The final 32 prompt queries are used as the observation window. Importance scores are smoothed by average pooling with kernel size 5, after which the highest-ranked prompt KV pairs are retained under the target budget.

LAQ. Starting from a temporary KV cache obtained by applying SnapKV to the original prompt cache, the target model performs up to eight lookahead decoding steps and stores the corresponding query states in the Q-Cache. These lookahead queries, together with the final eight prompt queries, are then used to recompute the attention-based importance scores of the original prompt KV entries before the second eviction.

SpecKV. The draft models are Qwen3-0.6B for Qwen3-8B, Llama-3.2-1B-Instruct for Llama-3.1-8B-Instruct, and mistral-7b-instruct-v0.3-bnb-4bit for Mistral-7B-Instruct-v0.3. The target-model queries corresponding to the prompt observation window and the generated draft tokens are used to rank historical prompt KV pairs. In the reproduction, the resulting importance scores are smoothed using average pooling with kernel size 5.

LookaheadKV. LookaheadKV is configured with 32 learnable lookahead tokens, and a separate set of lookahead modules is trained for each target model. The training data consist of 50K samples from ChatQA2-Long-SFT-data and 20K from tulu-3-sft-olmo-2-mixture. The predicted importance scores are smoothed using average pooling with kernel size 5 before prompt-KV selection.

Common Settings. All methods use random seed 42. Qwen3-8B, Llama-3.1-8B-Instruct, and Mistral-7B-Instruct-v0.3 are evaluated using their official checkpoints, tokenizers, and prompt templates. Greedy decoding with batch size one is used. All experiments are conducted on NVIDIA A100 80 GB GPUs.

C Additional Experiments on LongBench

In this section, we provide comprehensive experimental results on LongBench, a benchmark focused on long-context understanding, with input lengths ranging from 1235 to 18409 tokens. Tables 5, 6, and 7 present the detailed results of DeferKV and comparative methods on Qwen3-8B, Llama-3.1-8B-Instruct, and Mistral-7B-Instruct-v0.3, respectively, with cache budgets ranging from $64L$ to $1024L$. Overall, DeferKV achieves strong performance across models and cache budgets, with more pronounced improvements under constrained budgets. Taken together, these results further demonstrate the effectiveness and applicability of DeferKV across different models and long-context tasks.

D Compatibility with Existing KV Cache Compression Methods

We integrate DeferKV with PyramidKV, AdaKV, and ChunkKV and evaluate them on the 16 LongBench datasets using Qwen3-8B. Here, $B_{\text{total}} = nL$ denotes the aggregate prompt-KV budget, where L is the number of model layers. For PyramidKV and AdaKV, their original layer-wise and head-wise budget allocation strategies are retained, respectively. For ChunkKV, token-level scores are aggregated into chunk-level scores, while its chunk construction, selection, and index-reuse mechanisms remain unchanged. As shown in Table 8, DeferKV improves all three frameworks across the evaluated budgets.

E Additional Experiments on the RULER Benchmark

We evaluate Llama-3.1-8B-Instruct on all 11 RULER subtasks with $B_{\text{total}} = 1024L$ across context lengths ranging from 4K to 128K. Table 9 reports the detailed subtask performance and average accuracy at each context length.

F Results on Needle-in-a-Haystack

In this section, we further evaluate the effectiveness and robustness of DeferKV on the Needle-in-a-Haystack test using Mistral-7B-Instruct-v0.3. This benchmark assesses a model’s ability to retrieve critical information embedded within long contexts. We compare Full KV, SnapKV, LookaheadKV, and DeferKV under two cache budgets, $B_{\text{total}} = 128L$ and $B_{\text{total}} = 1024L$. Figures 6 and 7 present the retrieval results under these two settings, respectively. Under the constrained cache budget of $B_{\text{total}} = 128L$, DeferKV achieves a mean retrieval score of 0.892 and outperforms LookaheadKV. These results provide a more comprehensive comparison of DeferKV with existing methods across different cache budgets and further demonstrate its effectiveness in preserving KV pairs relevant to subsequent long-context retrieval.

G Numerical Analysis of Compression Delay Steps

The results reported in the main paper show that increasing the compression delay provides only modest performance improvements. Compared with $d = 1$, using $d = 2$ increases the task score from

	Single-Document QA			Multi-Document QA			Summarization			Few-Shot Learning			Synthetic		Code		
Method	NrtvQA	Qasper	MF-en	HotpotQA	2WikiMQA	MuSiQue	GovReport	QMSum	MultiNews	TREC	TriviaQA	SAMSum	PCount	PR-en	LCC	RB-P	Avg.
Qwen3-8B, $B_{\text{total}} = \text{FullKV}$																	
Full KV	24.73	48.05	54.39	58.16	43.79	34.27	33.45	24.12	24.84	71.50	90.21	44.56	9.50	100.00	69.34	65.48	49.77
Qwen3-8B, $B_{\text{total}} = 64L$																	
SnapKV	17.41	30.06	38.05	47.77	37.71	20.65	14.00	19.37	12.93	43.50	81.92	36.53	5.00	87.50	59.30	53.58	37.83
LAQ	18.99	37.88	43.31	<u>53.88</u>	<u>41.35</u>	23.24	16.18	19.40	<u>14.78</u>	<u>66.00</u>	<u>88.40</u>	37.64	5.50	94.50	<u>62.31</u>	<u>55.91</u>	<u>42.45</u>
SpecKV	<u>19.61</u>	29.97	49.79	51.03	36.97	22.14	13.27	19.72	14.82	37.50	73.63	36.71	<u>6.00</u>	<u>95.00</u>	45.15	46.62	37.37
LookaheadKV	18.22	32.27	40.12	47.66	38.01	<u>23.58</u>	14.61	18.36	13.18	46.95	83.29	36.69	5.00	90.40	59.06	54.28	38.86
DeferKV	21.87	<u>35.37</u>	<u>45.97</u>	56.99	41.88	31.20	<u>14.81</u>	<u>19.52</u>	13.52	70.50	89.98	<u>37.44</u>	8.00	97.50	62.52	56.47	43.97
Qwen3-8B, $B_{\text{total}} = 128L$																	
SnapKV	19.47	33.78	47.39	55.17	40.83	27.13	19.09	21.32	16.63	49.00	87.93	38.98	7.50	<u>99.00</u>	64.86	59.10	42.95
LAQ	20.70	<u>34.11</u>	<u>49.96</u>	<u>57.42</u>	41.54	29.75	20.10	<u>21.69</u>	<u>17.56</u>	<u>67.50</u>	88.65	<u>40.81</u>	7.50	<u>99.00</u>	<u>65.65</u>	59.18	<u>45.07</u>
SpecKV	<u>22.95</u>	32.00	53.73	55.35	40.46	29.26	22.58	22.40	19.15	64.50	87.60	34.45	4.50	<u>99.00</u>	53.08	52.10	43.32
LookaheadKV	21.13	33.91	48.34	57.39	<u>42.58</u>	<u>30.76</u>	<u>21.10</u>	21.02	16.88	60.64	<u>88.97</u>	39.24	<u>7.00</u>	100.00	65.92	<u>59.22</u>	44.63
DeferKV	23.84	38.51	49.70	57.51	43.01	33.30	19.40	21.10	17.18	71.50	90.05	41.43	<u>7.00</u>	100.00	65.01	59.30	46.12
Qwen3-8B, $B_{\text{total}} = 256L$																	
SnapKV	21.47	38.85	50.20	55.16	42.57	<u>33.18</u>	22.23	<u>22.52</u>	19.79	62.50	89.46	40.85	9.50	100.00	67.62	62.71	46.16
LAQ	22.41	<u>39.76</u>	<u>51.48</u>	57.22	42.67	32.41	23.67	22.18	<u>20.11</u>	69.00	89.58	<u>42.16</u>	<u>8.50</u>	100.00	67.11	<u>62.47</u>	<u>46.92</u>
SpecKV	25.42	38.90	54.18	<u>57.68</u>	42.48	28.94	26.39	23.52	22.44	<u>71.00</u>	89.28	34.11	2.50	99.00	56.67	54.64	45.45
LookaheadKV	22.76	39.21	50.74	57.46	<u>42.91</u>	33.06	<u>23.89</u>	22.06	20.03	67.50	<u>89.62</u>	41.78	8.00	<u>99.50</u>	<u>67.36</u>	62.18	46.75
DeferKV	<u>24.88</u>	41.72	50.80	58.21	42.96	33.39	22.70	22.27	19.91	71.50	90.05	43.62	9.50	100.00	66.81	61.47	47.49
Qwen3-8B, $B_{\text{total}} = 512L$																	
SnapKV	22.70	42.96	51.19	57.09	42.51	34.94	25.14	23.44	21.87	68.50	89.46	43.34	9.50	100.00	68.74	64.22	47.85
LAQ	23.16	43.58	<u>52.63</u>	56.16	43.02	34.36	25.46	22.02	21.94	<u>71.00</u>	90.05	43.35	9.50	100.00	67.92	<u>64.72</u>	48.05
SpecKV	<u>24.38</u>	43.27	53.93	<u>58.07</u>	42.90	31.48	29.51	<u>23.24</u>	24.05	<u>71.00</u>	89.96	29.60	<u>4.00</u>	<u>99.00</u>	56.84	56.85	46.13
LookaheadKV	23.10	<u>43.67</u>	52.21	56.86	<u>43.12</u>	<u>34.91</u>	25.46	22.71	22.08	<u>71.00</u>	<u>89.97</u>	43.85	9.50	<u>99.00</u>	<u>68.01</u>	64.98	<u>48.15</u>
DeferKV	24.82	44.78	51.96	58.09	43.18	33.83	<u>26.04</u>	22.74	<u>22.17</u>	71.50	90.05	<u>43.70</u>	9.50	100.00	67.55	64.02	48.37
Qwen3-8B, $B_{\text{total}} = 1024L$																	
SnapKV	24.49	45.20	53.14	58.09	42.89	35.09	28.48	23.51	23.75	69.00	90.30	43.03	8.50	100.00	<u>68.65</u>	65.82	48.75
LAQ	24.36	<u>45.42</u>	51.87	57.76	<u>43.41</u>	33.70	<u>28.99</u>	23.44	23.68	<u>71.44</u>	90.02	44.22	<u>9.65</u>	<u>99.65</u>	<u>68.63</u>	65.36	<u>48.85</u>
SpecKV	<u>25.03</u>	44.92	51.84	<u>57.97</u>	41.36	32.43	28.96	23.39	23.62	71.40	89.07	26.49	3.50	98.50	57.60	57.62	45.86
LookaheadKV	24.36	45.37	51.78	57.88	43.38	33.66	28.93	23.42	23.66	71.40	90.02	<u>44.31</u>	9.50	99.50	68.63	65.41	48.83
DeferKV	25.13	45.47	<u>51.94</u>	58.09	43.45	<u>33.76</u>	29.06	<u>23.49</u>	<u>23.72</u>	71.50	<u>90.05</u>	44.55	10.00	100.00	68.76	<u>65.60</u>	49.04

Table 5: Detailed LongBench results of Qwen3-8B under different prompt-KV budgets. The best and second-best results within each budget are shown in bold and underlined, respectively.

27.06 to 27.11, corresponding to a relative improvement of only 0.18%, while end-to-end latency increases from 11.562 to 11.618 seconds, or 0.48%. Therefore, waiting for a second decode query introduces an additional full-cache decoding step but produces only a negligible improvement in the final task score.

The same trade-off remains as the delay is increased further. At $d = 4$, the score improves by only 0.55% relative to $d = 1$, whereas latency increases by 1.63%. Peak memory remains nearly unchanged across the evaluated settings. These results suggest that additional decode observations can slightly refine KV importance estimation, but the improvement is not large enough to justify postponing compression. We therefore use $d = 1$, which incorporates real decode feedback at the earliest available step while avoiding unnecessary full-

cache decoding overhead.

H Effect of Stochastic Decoding

DeferKV uses the first real decode query to estimate prompt-KV importance. Since stochastic decoding may change the first generated token and its corresponding query representation, we further examine whether DeferKV remains effective under different decoding strategies. Specifically, we compare greedy decoding with sampling temperatures $T \in \{0.2, 0.8\}$ on LongBench using Qwen3-8B with $B_{\text{total}} = 128L$. The same prompts, cache budget, and method configurations are used across all decoding settings.

As shown in Table 10, sampling with a low temperature produces results close to those obtained with greedy decoding. Increasing the temperature to $T = 0.8$ leads to a moderate performance de-

Method	Single-Document QA			Multi-Document QA			Summarization			Few-Shot Learning			Synthetic		Code		Avg.
	NrtvQA	Qasper	MF-en	HotpotQA	2WikiMQA	MuSiQue	GovReport	QMSum	MultiNews	TREC	TriviaQA	SAMSum	PCount	PR-en	LCC	RB-P	
Llama-3.1-8B-Instruct, $B_{\text{total}} = \text{FullKV}$																	
Full KV	31.20	46.66	56.64	58.10	48.51	31.57	34.37	25.18	27.14	73.00	91.65	43.80	7.64	99.50	65.18	58.90	49.94
Llama-3.1-8B-Instruct, $B_{\text{total}} = 64L$																	
SnapKV	27.32	25.25	44.30	53.98	42.48	27.10	17.88	22.20	17.45	41.00	85.37	37.72	<u>8.50</u>	100.00	55.35	48.69	40.91
LAQ	27.66	<u>28.03</u>	47.05	58.12	<u>45.06</u>	<u>30.60</u>	18.85	22.51	<u>19.01</u>	71.50	90.02	40.16	<u>8.50</u>	<u>99.50</u>	56.39	<u>51.29</u>	<u>44.64</u>
SpecKV	<u>29.31</u>	23.34	56.27	56.81	41.91	28.70	21.22	24.43	18.66	46.98	76.62	39.69	9.71	100.00	41.67	42.44	41.11
LookaheadKV	27.95	26.61	47.23	54.93	44.90	30.10	19.24	22.01	18.34	<u>70.00</u>	88.65	<u>40.21</u>	<u>8.50</u>	99.00	<u>56.54</u>	49.03	43.95
DeferKV	29.83	31.02	<u>49.95</u>	<u>57.55</u>	47.65	30.77	<u>20.68</u>	<u>22.77</u>	19.29	71.50	<u>88.95</u>	40.41	7.88	100.00	57.98	51.56	45.49
Llama-3.1-8B-Instruct, $B_{\text{total}} = 128L$																	
SnapKV	26.58	31.30	48.78	55.16	46.06	27.80	21.17	23.47	<u>21.21</u>	48.00	89.38	40.14	8.50	<u>99.50</u>	59.32	52.32	43.67
LAQ	28.47	<u>32.84</u>	50.12	<u>58.21</u>	46.72	30.91	21.36	23.25	21.02	<u>70.50</u>	<u>90.84</u>	<u>41.22</u>	8.34	<u>99.50</u>	59.45	<u>53.02</u>	<u>45.99</u>
SpecKV	<u>29.54</u>	29.18	56.41	57.39	43.76	28.61	23.84	24.88	21.15	55.00	82.73	37.42	<u>8.44</u>	<u>99.50</u>	46.90	45.86	43.16
LookaheadKV	28.81	32.47	50.46	57.03	<u>46.91</u>	<u>30.84</u>	22.01	23.05	20.72	69.50	90.15	41.04	8.32	<u>99.50</u>	<u>59.68</u>	52.74	45.83
DeferKV	30.63	35.65	<u>52.51</u>	58.47	48.09	30.67	<u>22.92</u>	<u>23.66</u>	21.97	72.00	91.79	41.61	8.21	100.00	60.08	53.39	46.98
Llama-3.1-8B-Instruct, $B_{\text{total}} = 256L$																	
SnapKV	27.19	36.26	52.52	57.50	46.19	29.04	23.52	<u>24.16</u>	22.57	56.50	90.63	40.95	8.25	99.50	62.24	55.38	45.78
LAQ	<u>29.68</u>	<u>39.17</u>	53.31	58.22	47.91	<u>30.90</u>	24.85	23.92	<u>23.48</u>	<u>71.50</u>	91.42	<u>42.19</u>	<u>7.96</u>	99.50	<u>62.68</u>	<u>55.92</u>	<u>47.66</u>
SpecKV	29.08	36.24	56.58	<u>58.41</u>	47.02	28.49	27.28	25.64	24.76	65.00	88.72	33.84	5.61	<u>99.00</u>	49.57	47.36	45.16
LookaheadKV	29.54	38.54	53.66	58.07	<u>48.16</u>	30.89	<u>24.91</u>	23.71	23.11	<u>71.50</u>	<u>91.53</u>	41.96	7.88	99.50	62.84	56.06	47.62
DeferKV	29.71	40.28	<u>54.12</u>	58.71	48.36	30.93	24.72	23.91	23.28	72.50	91.86	42.42	7.58	99.50	62.13	55.10	47.82
Llama-3.1-8B-Instruct, $B_{\text{total}} = 512L$																	
SnapKV	29.92	40.20	55.21	57.85	48.02	30.03	25.79	<u>24.40</u>	24.33	65.00	91.95	41.80	8.08	<u>99.50</u>	64.39	<u>57.27</u>	47.73
LAQ	<u>30.58</u>	<u>42.99</u>	54.46	58.13	48.40	30.79	26.45	24.15	24.51	<u>70.96</u>	91.51	<u>42.74</u>	7.59	<u>99.50</u>	<u>64.08</u>	56.78	<u>48.35</u>
SpecKV	28.23	41.70	56.69	59.29	<u>48.62</u>	28.24	30.41	26.44	27.08	72.00	91.63	28.51	3.23	99.00	53.07	49.99	46.51
LookaheadKV	30.11	41.12	<u>55.34</u>	57.98	49.23	31.34	26.02	23.85	24.06	72.00	<u>91.85</u>	41.64	<u>7.75</u>	<u>99.50</u>	63.21	57.33	48.27
DeferKV	30.70	43.91	54.84	<u>58.54</u>	48.51	<u>31.02</u>	<u>26.60</u>	24.22	<u>24.53</u>	72.00	91.48	42.98	7.47	100.00	63.90	56.70	48.59
Llama-3.1-8B-Instruct, $B_{\text{total}} = 1024L$																	
SnapKV	29.81	44.00	<u>55.91</u>	57.67	48.01	30.77	27.77	24.50	25.79	67.50	91.90	42.95	7.42	99.50	<u>64.59</u>	58.30	48.52
LAQ	30.74	<u>45.01</u>	55.62	58.17	48.43	31.19	<u>29.14</u>	<u>24.52</u>	<u>25.88</u>	71.50	91.64	43.32	<u>7.61</u>	99.50	64.68	58.05	<u>49.06</u>
SpecKV	29.66	43.88	56.81	58.92	48.08	28.92	31.92	26.70	27.22	72.50	91.61	28.92	3.41	<u>99.00</u>	54.21	51.06	47.05
LookaheadKV	30.46	44.72	55.74	58.03	48.91	<u>31.36</u>	28.68	24.41	25.67	<u>72.00</u>	<u>91.73</u>	43.06	7.69	99.50	64.55	<u>58.18</u>	49.04
DeferKV	<u>30.53</u>	45.43	55.82	<u>58.38</u>	<u>48.63</u>	31.40	29.02	24.27	<u>25.88</u>	72.50	91.48	<u>43.28</u>	7.53	99.50	64.38	57.86	49.12

Table 6: Detailed LongBench results of Llama-3.1-8B-Instruct under different prompt-KV budgets. The best and second-best results within each budget are shown in bold and underlined, respectively.

crease across all methods, reflecting the greater variability introduced during generation. Nevertheless, DeferKV maintains strong performance relative to the other KV compression methods under both temperatures. These results indicate that its improvement is not limited to deterministic decoding and that the decode-aware importance estimates remain effective when the first generated token is sampled.

	Single-Document QA			Multi-Document QA			Summarization			Few-Shot Learning			Synthetic		Code		
Method	NrivQA	Qasper	MF-en	HotpotQA	2WikiMQA	MusiQue	GovReport	QMSum	MultiNews	TREC	TriviaQA	SAMSum	PCount	PR-en	LCC	RB-P	Avg.
Mistral-7B-Instruct-v0.3, $B_{\text{total}} = \text{FullKV}$																	
Full KV	27.01	37.97	50.14	51.69	39.17	28.02	34.27	25.66	26.62	76.00	88.59	47.50	7.00	98.00	61.53	62.59	47.61
Mistral-7B-Instruct-v0.3, $B_{\text{total}} = 64L$																	
SnapKV	17.65	22.08	36.24	41.95	33.98	20.89	18.83	21.14	16.34	38.50	86.96	39.81	4.50	77.00	51.57	49.59	36.06
LAQ	19.72	<u>28.68</u>	40.14	<u>48.46</u>	<u>35.96</u>	<u>23.44</u>	<u>20.82</u>	21.47	18.93	<u>70.58</u>	89.15	42.32	4.75	79.80	<u>53.08</u>	<u>51.19</u>	<u>40.53</u>
SpecKV	<u>19.74</u>	21.08	48.10	45.00	33.32	22.44	22.64	22.93	17.89	38.49	78.44	40.89	<u>5.60</u>	<u>80.75</u>	37.66	42.98	36.12
LookaheadKV	19.48	24.28	38.88	42.86	34.76	23.22	<u>20.82</u>	21.02	18.93	57.38	88.09	41.11	4.50	78.16	51.92	49.93	38.46
DeferKV	26.11	28.83	<u>43.02</u>	49.10	36.87	25.15	20.49	<u>21.75</u>	<u>18.50</u>	73.50	<u>89.04</u>	<u>42.17</u>	7.50	81.00	53.41	51.46	41.74
Mistral-7B-Instruct-v0.3, $B_{\text{total}} = 128L$																	
SnapKV	22.56	26.89	40.86	44.90	33.63	22.74	21.91	22.07	20.66	45.00	88.74	43.28	5.50	90.50	56.69	<u>55.62</u>	40.10
LAQ	<u>24.58</u>	<u>29.63</u>	43.85	<u>49.82</u>	<u>35.74</u>	24.52	22.34	22.41	<u>21.13</u>	<u>70.80</u>	<u>89.24</u>	<u>44.12</u>	4.75	<u>91.50</u>	56.92	55.89	<u>42.95</u>
SpecKV	23.84	26.17	49.18	47.23	34.52	23.06	24.57	24.13	22.35	52.75	84.16	39.12	<u>5.25</u>	91.00	42.85	47.60	39.86
LookaheadKV	24.10	28.96	43.72	47.66	35.58	<u>24.70</u>	22.48	22.26	21.04	65.40	88.98	43.86	4.50	91.00	56.74	55.41	42.27
DeferKV	26.51	30.44	<u>47.14</u>	50.79	36.92	25.88	<u>22.82</u>	<u>22.55</u>	21.08	75.50	89.29	44.33	4.00	92.00	<u>56.87</u>	55.25	43.84
Mistral-7B-Instruct-v0.3, $B_{\text{total}} = 256L$																	
SnapKV	24.97	30.44	47.47	48.25	36.16	24.45	23.48	23.51	22.60	57.00	89.01	43.60	5.00	<u>96.00</u>	58.68	58.14	43.05
LAQ	<u>25.62</u>	32.94	48.76	<u>50.21</u>	37.16	26.32	24.92	<u>23.68</u>	<u>23.01</u>	<u>74.50</u>	<u>89.18</u>	46.33	5.50	96.50	59.21	58.72	<u>45.16</u>
SpecKV	25.11	31.82	51.26	49.35	36.24	23.94	27.21	25.02	24.31	64.00	87.63	35.84	4.25	95.75	46.28	51.32	42.46
LookaheadKV	25.49	32.37	48.94	49.88	37.51	<u>26.91</u>	24.74	23.43	22.91	73.20	89.12	45.87	<u>5.25</u>	<u>96.00</u>	<u>59.04</u>	<u>58.46</u>	44.95
DeferKV	26.44	<u>32.55</u>	<u>49.32</u>	50.96	<u>37.44</u>	27.34	<u>24.96</u>	23.27	22.78	76.00	89.24	<u>46.00</u>	5.50	95.50	58.66	57.80	45.24
Mistral-7B-Instruct-v0.3, $B_{\text{total}} = 512L$																	
SnapKV	25.48	33.15	<u>49.37</u>	48.98	36.54	24.68	25.61	<u>24.27</u>	24.11	67.50	<u>89.36</u>	44.82	6.00	97.50	60.36	<u>61.28</u>	44.94
LAQ	<u>26.23</u>	<u>34.50</u>	48.82	49.95	38.22	26.58	26.10	24.10	24.20	74.66	89.08	<u>47.10</u>	6.00	97.50	<u>60.02</u>	61.44	<u>45.91</u>
SpecKV	25.48	34.10	51.48	<u>50.19</u>	37.03	22.06	30.10	25.69	25.58	72.25	89.45	31.30	<u>3.83</u>	<u>96.75</u>	48.75	53.96	43.63
LookaheadKV	25.79	33.86	49.15	48.94	39.16	<u>26.67</u>	25.88	24.12	23.93	<u>75.29</u>	89.20	46.33	6.00	97.50	59.54	61.44	45.80
DeferKV	26.90	35.37	48.91	51.10	<u>38.72</u>	27.63	<u>26.44</u>	24.13	<u>24.27</u>	76.00	89.07	47.18	6.00	97.50	59.68	60.49	46.21
Mistral-7B-Instruct-v0.3, $B_{\text{total}} = 1024L$																	
SnapKV	27.14	35.88	49.18	50.02	37.63	26.03	27.42	<u>25.06</u>	25.35	72.50	89.11	46.18	4.00	98.50	61.33	61.75	46.07
LAQ	26.72	<u>36.31</u>	<u>49.66</u>	<u>51.32</u>	38.51	<u>27.11</u>	28.71	24.99	25.62	75.40	89.17	<u>47.36</u>	<u>6.25</u>	<u>98.00</u>	<u>61.18</u>	61.41	<u>46.73</u>
SpecKV	26.50	35.46	51.62	51.06	37.88	23.24	32.16	26.19	26.44	74.00	89.38	32.20	4.00	97.50	50.13	55.26	44.56
LookaheadKV	<u>26.85</u>	35.82	49.54	51.15	<u>38.70</u>	27.05	28.17	24.64	25.28	<u>75.70</u>	<u>89.22</u>	47.08	<u>6.25</u>	<u>98.00</u>	60.98	<u>61.58</u>	46.63
DeferKV	26.37	36.88	49.38	51.84	38.76	27.12	<u>28.73</u>	24.76	<u>25.67</u>	76.00	89.19	47.50	6.50	<u>98.00</u>	61.14	61.03	46.80

Table 7: Detailed LongBench results of Mistral-7B-Instruct-v0.3 under different prompt-KV budgets. The best and second-best results within each budget are shown in bold and underlined, respectively.

Method	Prompt-KV Budget B_{total}					Avg.
	64L	128L	256L	512L	1024L	
PyramidKV	38.30	43.25	46.40	48.00	48.88	44.97
DeferKV w. PyramidKV	44.18	46.30	47.66	48.52	49.18	47.17 ($\uparrow$ 4.89%)
AdaKV	38.55	43.50	46.58	48.08	48.92	45.13
DeferKV w. AdaKV	44.30	46.38	47.72	48.57	49.22	47.24 ($\uparrow$ 4.68%)
ChunkKV	37.60	42.80	46.05	47.78	48.68	44.58
DeferKV w. ChunkKV	44.05	46.22	47.58	48.46	49.12	47.09 ($\uparrow$ 5.63%)

Table 8: Compatibility results on LongBench using Qwen3-8B. Each integrated variant retains the allocation and selection mechanisms of the corresponding compression method while replacing its prompt-only importance scores with DeferKV scores.

Method	S-NIAH-1	S-NIAH-2	S-NIAH-3	MK-NIAH-1	MK-NIAH-2	MK-NIAH-3	MQ-NIAH	MV-NIAH	CWE	FWE	VT	Avg. Acc.
Llama-3.1-8B-Instruct, $B_{\text{total}} = 1024L$, context length = $4K$												
Full KV	100.00	100.00	99.60	100.00	100.00	99.60	99.90	96.25	99.78	97.67	99.96	99.34
SnapKV	100.00	99.00	20.60	<u>99.60</u>	93.00	31.00	99.20	90.75	91.56	95.31	99.96	83.63
LAQ	100.00	<u>99.10</u>	<u>53.90</u>	<u>99.60</u>	<u>96.20</u>	<u>36.80</u>	<u>99.60</u>	<u>93.37</u>	88.25	94.90	<u>99.90</u>	<u>87.42</u>
SpecKV	100.00	99.00	42.60	<u>99.50</u>	95.20	34.60	99.58	92.51	<u>89.65</u>	<u>95.00</u>	<u>99.90</u>	86.14
LookaheadKV	100.00	99.00	47.30	<u>99.60</u>	95.60	35.50	<u>99.60</u>	92.82	88.94	<u>95.00</u>	<u>99.90</u>	86.66
DeferKV	100.00	100.00	99.80	100.00	100.00	45.80	99.95	97.05	82.66	94.73	99.96	92.72
Llama-3.1-8B-Instruct, $B_{\text{total}} = 1024L$, context length = $8K$												
Full KV	100.00	100.00	100.00	100.00	99.80	98.80	100.00	95.75	97.62	95.27	99.92	98.83
SnapKV	100.00	98.40	12.00	98.00	85.20	7.80	97.60	87.42	56.13	88.52	99.56	75.51
LAQ	100.00	<u>98.10</u>	<u>63.50</u>	<u>99.00</u>	<u>94.00</u>	6.10	<u>98.90</u>	<u>92.84</u>	<u>58.61</u>	<u>87.83</u>	<u>99.90</u>	<u>81.78</u>
SpecKV	100.00	98.70	44.50	<u>98.50</u>	<u>90.90</u>	<u>6.60</u>	<u>98.45</u>	<u>90.72</u>	57.69	88.10	<u>99.57</u>	79.43
LookaheadKV	100.00	98.90	54.60	98.80	92.50	6.40	98.68	91.86	58.17	87.98	<u>99.70</u>	80.69
DeferKV	100.00	100.00	99.60	100.00	99.80	5.60	99.70	96.95	59.94	87.73	99.92	86.29
Llama-3.1-8B-Instruct, $B_{\text{total}} = 1024L$, context length = $16K$												
Full KV	100.00	100.00	100.00	99.60	100.00	99.00	99.85	98.25	90.90	96.67	99.80	98.55
SnapKV	100.00	97.00	4.00	97.60	74.00	3.80	97.25	88.95	27.36	92.40	<u>99.60</u>	71.09
LAQ	100.00	<u>98.40</u>	<u>56.80</u>	<u>98.50</u>	<u>88.50</u>	2.20	<u>98.93</u>	<u>94.25</u>	<u>28.94</u>	<u>94.85</u>	<u>99.60</u>	<u>78.27</u>
SpecKV	100.00	97.80	37.20	98.10	83.30	<u>2.60</u>	98.30	92.30	28.50	93.90	<u>99.60</u>	75.60
LookaheadKV	100.00	98.10	47.30	98.30	85.90	2.40	98.65	93.36	28.77	94.40	<u>99.60</u>	76.98
DeferKV	100.00	100.00	97.40	99.60	99.40	1.60	99.90	98.55	29.70	96.93	99.76	83.89
Llama-3.1-8B-Instruct, $B_{\text{total}} = 1024L$, context length = $32K$												
Full KV	100.00	100.00	100.00	100.00	100.00	99.60	99.90	98.95	48.60	97.07	99.68	94.89
SnapKV	100.00	96.20	6.20	96.40	61.00	2.00	92.50	87.24	16.96	71.23	97.64	66.12
LAQ	100.00	<u>98.10</u>	<u>58.10</u>	<u>98.10</u>	<u>78.30</u>	0.50	<u>96.78</u>	<u>93.77</u>	<u>18.29</u>	<u>75.16</u>	<u>97.89</u>	<u>74.09</u>
SpecKV	100.00	97.10	35.30	97.30	70.70	<u>1.00</u>	94.87	90.76	17.78	73.35	97.67	70.53
LookaheadKV	100.00	97.60	47.70	97.70	74.80	0.70	95.86	92.34	17.97	74.32	97.74	72.43
DeferKV	100.00	100.00	98.00	99.80	91.20	0.00	99.85	99.00	18.80	78.60	98.16	80.31
Llama-3.1-8B-Instruct, $B_{\text{total}} = 1024L$, context length = $64K$												
Full KV	100.00	100.00	99.80	99.80	99.20	94.00	99.75	98.95	7.96	90.60	98.32	89.85
SnapKV	100.00	95.40	3.20	97.00	32.40	0.20	90.40	54.45	2.89	53.43	89.50	56.26
LAQ	100.00	<u>97.90</u>	<u>58.10</u>	<u>98.50</u>	<u>64.80</u>	1.50	<u>96.14</u>	<u>81.37</u>	3.56	<u>58.09</u>	<u>90.57</u>	<u>68.23</u>
SpecKV	100.00	96.70	33.50	97.80	50.30	0.80	93.55	69.16	3.34	55.90	90.08	62.83
LookaheadKV	100.00	97.30	46.70	98.20	58.10	1.20	94.94	75.68	3.41	57.11	90.39	65.73
DeferKV	100.00	100.00	92.80	99.80	85.20	3.00	99.50	98.70	<u>3.42</u>	61.07	91.28	75.89
Llama-3.1-8B-Instruct, $B_{\text{total}} = 1024L$, context length = $128K$												
Full KV	97.40	97.80	95.20	96.20	87.00	63.20	95.85	94.95	0.06	64.73	80.08	79.32
SnapKV	93.60	93.80	1.40	85.80	25.60	0.00	76.80	27.00	0.08	26.49	71.76	45.67
LAQ	<u>96.50</u>	<u>97.00</u>	<u>55.90</u>	<u>92.90</u>	<u>63.40</u>	0.90	<u>89.84</u>	<u>73.31</u>	<u>1.43</u>	<u>36.28</u>	<u>75.20</u>	<u>62.06</u>
SpecKV	95.20	95.30	29.90	89.40	45.40	0.30	83.55	50.93	0.86	31.51	73.52	54.17
LookaheadKV	96.00	96.40	46.70	91.70	57.00	0.60	87.63	65.39	1.26	34.57	74.61	59.26
DeferKV	97.40	98.80	78.40	96.20	79.00	1.80	95.25	93.10	1.68	40.87	76.96	69.04

Table 9: Detailed RULER results across context lengths from 4K to 128K using Llama-3.1-8B-Instruct with $B_{\text{total}} = 1024L$.

Decoding	Full KV	SnapKV	LAQ	SpecKV	LookaheadKV	DeferKV
Greedy	49.77	42.95	45.07	43.32	44.63	46.12
$T = 0.2$	49.64	42.07	44.58	40.94	43.02	46.01
$T = 0.8$	48.91	41.34	43.82	40.16	42.28	45.32

Table 10: Average LongBench performance under greedy and stochastic decoding using Qwen3-8B with $B_{\text{total}} = 128L$.

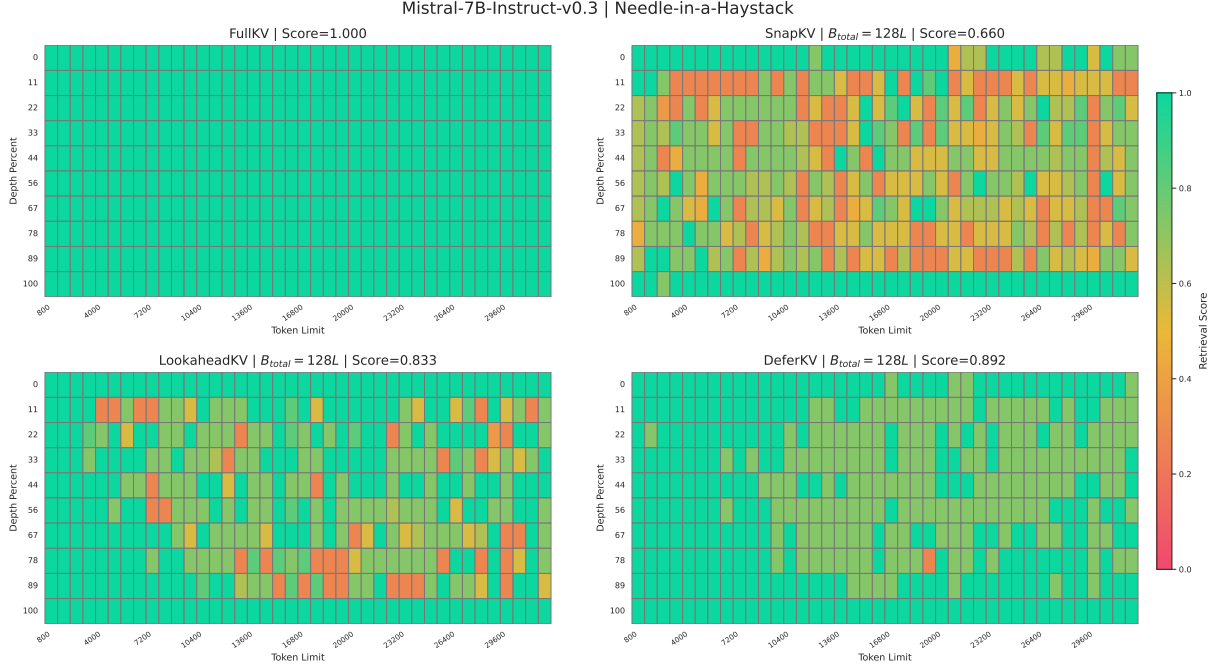


Figure 6: Needle-in-a-Haystack retrieval performance of Full KV, SnapKV, LookaheadKV, and DeferKV on Mistral-7B-Instruct-v0.3 with $B_{\text{total}} = 128L$.

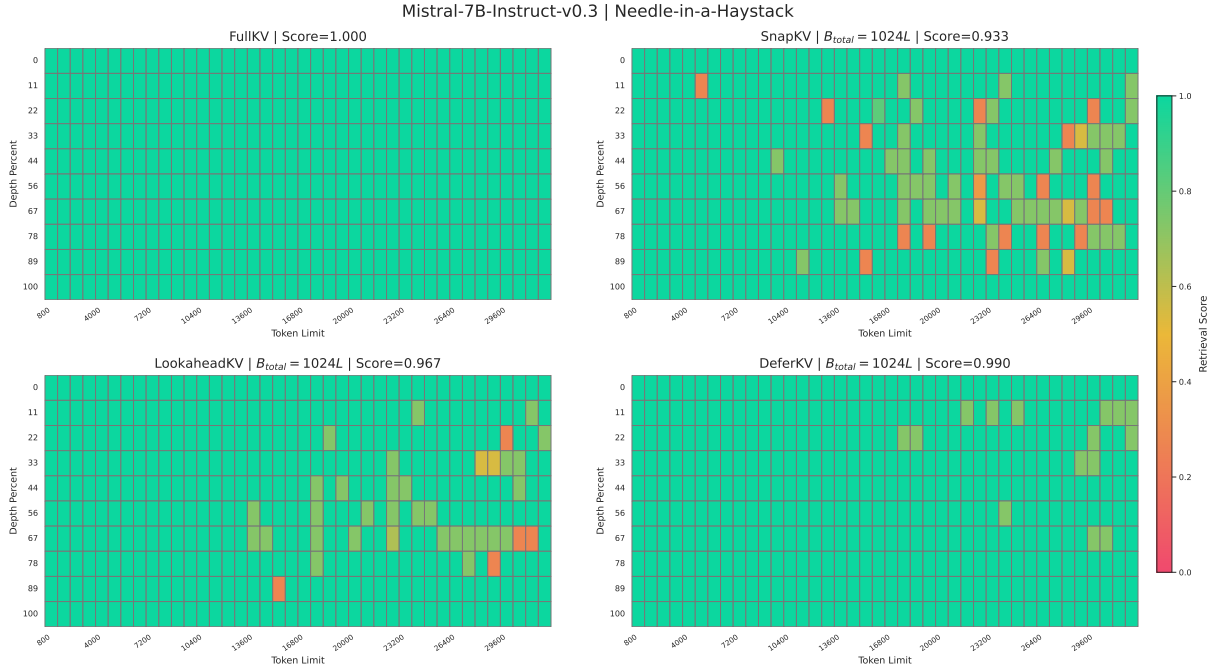


Figure 7: Needle-in-a-Haystack retrieval performance of Full KV, SnapKV, LookaheadKV, and DeferKV on Mistral-7B-Instruct-v0.3 with $B_{\text{total}} = 1024L$.